Operators in Python

Housekeeping

String Operators

- Addition operator (+) – called concatenation

- You can add two strings together using the addition operator

```
a = 'Hel'
b = 'lo'
output = a+b
print(output)
```

- Multiplication operator (*) – called repetition

- You can multiply a string by an INTEGER number

```
a = 'Hel'
b = 'lo'
output = a+b
print(output*3)
```

You can use the following operators on strings:

- >
- <
- >=
- <=
- ==
- !=

You can not use:

- Subtraction

/ Division

We will look at this in more detail later.

Variable assignment

- A variable can only link to one value at a time

```
1 a = '123'
2 a = int(a)
3
4 b= '123'
5 c= int(123)
```

Multiple Assignments

- You can assign the same value to multiple variables at the same time

```
a = b = c = 10
```

Multiple Assignments

- You can assign the multiple values to multiple variables at the same time

```
x,y,z = 10,20,30


#The three values on the right go into memory from
 left to right
#The variables are assigned to the values from left
 to right
```

Swapping values – You must learn this

```
x = 10
y = 5

print(x,y)

x,y = y,x

print(x,y)
```

Swapping values – You must learn this

```
1  x = 10
2  y = 5
3  print(x,y)
4
5  temp = x
6  x = y
7  y = temp
8
9  print(x,y)
```

print(statement) - separator

When you use the print function with comma separated values. The print statement replaces the comma with a space.

```
print('My','name','is','bob’)
```

By default each word is separated with a space.

```
>>> %Run -c $EDITOR_CONTENT
My name is bob
```

print(statement) - separator

You can change this default value using the sep argument inside the print function.

```
print('My','name','is','bob', sep = '...’)
```

Now each word will be separated by an ellipsis

```
>>> %Run -c $EDITOR_CONTENT
My...name...is...bob
```

print(statement) - end

The print statement automatically adds a newline character at the end of each print statement.

```
print('Hello')
print('Bob’)
```

```
>>> %Run -c $EDITOR_CONTENT
 Hello
 Bob
```

print(statement) - end

You can change this using the end argument:

```
print('Hello', end = ' ')
print('Bob')
```

```
>>> %Run -c $EDITOR_CONTENT
 Hello Bob
```